

SG-Lean 思路文档（第二版）

复用导向的轻量自博弈证明器

实现规格

本文件的定位：这是实现规格，不是理念介绍。任何人拿着它就应该能直接开工，不需要再问“这一步的代码写成什么样”。

与第一版的关系：第一版《SG-Lean 思路汇报》保留作为理念记录；本版是它的修订与细化，凡两者冲突以本版为准。文献基线与“哪些已被别人做过”见《库的相关工作》。

阅读顺序：第 1 节（目标与指标）与第 2 节（总览）必须读；写代码时按第 3 至 5 节的步骤规格与第 9 节的阶段计划执行；接口字段见附录 A，函数签名见附录 B。

目录

1	目标与评判指标	3
1.1	产品目标	3
1.2	三个评判指标	3
1.3	不可违反的研究约束	4
2	系统总览	5
2.1	两个时间尺度	5
2.2	三个角色与代码归属	5
2.3	四类数据与不相交要求	6
3	单题求解流程（在线）	6
3.1	步骤 1：输入解析	6
3.2	步骤 2：门检	6
3.3	步骤 3：廉价 tactic 兜底	7
3.4	步骤 4：分层检索	7
3.5	步骤 5：求解（生成候选证明）	7
3.6	步骤 6：内核验证	8
3.7	步骤 7：repair 定向重试	8
3.8	步骤 8：输出与记账	8
3.9	步骤 9：预算分配与停止条件	8

4	求解器规格	9
4.1	默认配置（写死在配置文件里）	9
4.2	求解器固定吗：分两层回答	9
4.3	提示词规格	9
4.4	输出解析规则	10
4.5	成本模型	10
4.6	换模型的影响（写在文档里，避免临时争论）	10
5	引理库的建立与管理（离线）	11
5.1	一轮闭环的伪代码	11
5.2	需求挖掘	11
5.3	出题者	11
5.4	硬门	12
5.5	复用判据（本项目的核心）	12
5.5.1	要解决的问题	12
5.5.2	定义	12
5.5.3	选择算法	13
5.5.4	淘汰规则	13
5.6	物化与入库	14
6	有没有要训练的小规模参数	14
7	数据集规格	15
7.1	来源与用途	15
7.2	C 的来源与同源检查	15
7.3	难度标定流程（必须先做）	15
7.4	测试集 T 是什么	16
7.5	防泄漏检查清单（每次正式实验前逐条打勾）	16
8	实验设计	16
8.1	三组对照	16
8.2	指标与公式	17
8.3	样本量与统计功效（先算再跑）	17
8.4	预算与墙钟估算	17
8.5	报告规范	18
9	分阶段实施计划	18
9.1	阶段 P1：证明器本体	18
9.2	阶段 P2：小实验台	18
9.3	阶段 P3：三组对照	20
9.4	阶段 P4：成本与复用分析	20
9.5	阶段 P5：写作	20

10 风险与降级预案	21
A 数据结构（字段级）	21
A.1 运行记录 RunRecord	21
A.2 轨迹记录 Trace	21
A.3 需求记录 Demand	22
A.4 库条目 LibraryEntry	22
B 代码骨架（函数签名清单）	22
C 术语与缩写	23

1 目标与评判指标

1.1 产品目标

构建一个 Lean 4 形式化数学证明器：输入一条数学命题的 Lean 形式化陈述，输出一段通过 Lean 内核检验的 tactic 证明脚本。

输入	两种形式之一：(i) 一个 <code>.lean</code> 文件，其中含 <code>theorem</code> 或 <code>example</code> 语句且证明体为 <code>sorry</code> ；(ii) 一条闭式命题字符串（只引用全局常量，变量用全称量词引入）。
输出	一个 JSON 对象（或写回 <code>.lean</code> 文件）：通过的证明脚本、命中的求解路径、验证记录、耗时与 token 成本。
保证	输出中的每一篇证明都经过 Lean 内核终检（含无 <code>sorry</code> 、无残余元变量的检查）。未通过验证的候选不出现在输出中。

1.2 三个评判指标

本项目只用三个指标衡量，并且优先级固定如下。任何取舍、任何技术选择、任何“要不要做某件事”的判断，都按这个顺序裁决。

成本的三个量（必须同时报）：

$$\text{CostPerSolved} = \frac{\text{总 token 数}}{\text{解出题数}}, \quad \text{CostPerLemma} = \frac{\text{建库总 token 数}}{\text{入库引理数}},$$

$$\text{CostPerReusable} = \frac{\text{建库总 token 数}}{\#\{\ell \in L : \text{reuse}(\ell) \geq 2\}}$$

其中 $\text{reuse}(\ell)$ 是引理 ℓ 被多少个不同目标的通过验收的证明实际引用过，精确定义见 5.5 节。
 CostPerReusable 是本项目最看重的成本量：它把“复用”和“花钱”放在同一个分母里。

轻量化的硬约束清单（全部必须满足）：

优先级如何裁决冲突（写死，避免临时争论）：

1. 若两个方案正确率有显著差异，选正确率高的，成本不计较。

优先级	指标	定义	计算公式
1	正确率	在冻结的测试集上，目标级通过率。一道题只要有至少一篇候选通过内核检验即算解出。	$\text{pass}@k = \frac{1}{ T } \sum_{w \in T} \mathbf{1}[\max_{i \leq k} \text{verify}(w, y_i) = 1]$
2	成本	为达到上述正确率所付出的模型调用成本。	见下方三个成本量
3	轻量化	不是优化对象，是可交付性的硬约束：违反即判定为不合格，无论正确率多高。	见下方约束清单

约束	上限	说明
可训练参数	0	不做任何梯度更新，不训练、不微调、不训练检索器
GPU 需求	0	全部实验在单机 CPU + API 上完成
单次实验墙钟	≤ 12 小时	一次完整的建库加评测；可分批跨日执行
单次实验 API 预算	≤ 500 元	全部实验合计 ≤ 3000 元
复现门槛	一条命令	提供一键复现脚本，入口不带交互

2. 若正确率无显著差异（置信区间重叠），选 `CostPerSolved` 低的。
3. 若成本也接近，选实现更简单、复现更容易的那个。
4. 任何方案若违反轻量化硬约束，直接淘汰，无论正确率多高。

1.3 不可违反的研究约束

1. 数据角色隔离：建库集 C 与报告集 T 必须不同源；调参集 D 可反复使用； T 在框架冻结后只运行一次。
2. 绝不输出未验证的证明。
3. 不提交密钥：`sgsr/models/.env` 不入库。
4. 报告可回溯：每个正式数字必须能由 `experiments/results/` 下的文件复现；文件名带 `mode` 与规模后缀。
5. 库版本冻结：评测时库不再更新；在线生成的临时引理不进全局库。

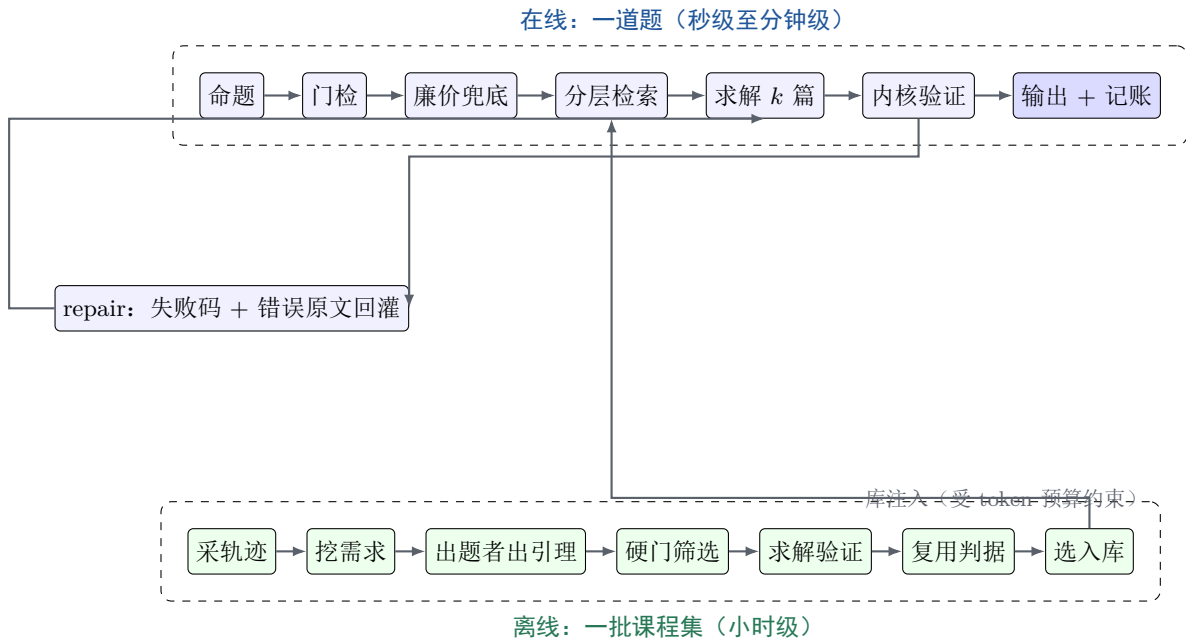


图 1: 两个时间尺度：在线求解与离线建库，只通过”库”这一个接口相连。

2 系统总览

2.1 两个时间尺度

两个尺度只通过一个接口相连：库。在线只有求解者出场；出题者与评审者只存在于离线。

2.2 三个角色与代码归属

角色	担当者	代码位置与状态
求解者	语言模型（API），整篇生成证明	sgsr/models/proxy.py 的 /solve; 已实现
出题者	语言模型（API），条件化生成引理候选	sgsr/models/prompts.py 的 conjecture_prompt; sgsr/pipeline/conjecture.py; 已实现，需扩展条件化信号
评审者（Guide）	形式化判据：硬门 + 复用判据；不用语言模型	硬门已实现（SgsLean/ 下四个模块）；复用判据待实现
（对照组）	SGS 原始的 LLM 打分 rubric	/guide + GuideClient + rubric, 完整保留，作为实验 A 组

符号	角色	用量	来源要求
C	建库：采轨迹、挖需求、出引理	100 至 300 道	来源甲；与 D, T 不同源
D	调参：定 k 、阈值、库容；看方向	50 至 150 道	来源乙
T	最终报告	50 至 150 道	来源乙，与 D 不相交；只跑一次
W_{sel}	复用判据的测量与选择	$= C$ 的一个子集	只能是 C

2.3 四类数据与不相交要求

为什么必须两个来源：如果建库用的题与报告的题来自同一份数据，那么”库对报告集的增益”里就混进了”库见过这些题”的成分，测试集信息泄漏，结论作废。这是我们此前踩过的真实错误，处理方案写在 `sgs-reap/docs/data-protocol.md`。

3 单题求解流程（在线）

以下每一步都给出：模块、输入、（字段级）输出、算法要点、失败处理、成本记账。

3.1 步骤 1：输入解析

模块：sgsr/pipeline/prover.py 的 parse_input

输入：.lean 文件路径或命题字符串

输出：{"stmt": str, "origin": str, "binders": []}

算法：定位 theorem 或 example 语句，取语句名与绑定变量，把 $(x : T) (h : P)$ 形式改写为全称量化 forall $x : T, P \rightarrow \dots$ （协议 v1.2 要求闭式命题，不支持局部上下文透传）。

失败处理：无法抽取则记 parse_error 并返回；这属于输入问题，不是”证不出来”，不得计入失败率。

成本：0。

3.2 步骤 2：门检

模块：sgslean/SgsLean/Gate.lean，经 Lean 服务调用

输入：stmt

输出：{"ok": bool, "reason": str, "isProp": bool}

算法：把语句 elaborate 成类型表达式，检查它是不是 Prop。

失败处理：ok=false 则终止并记录，不计入解出率分母之外的分母。

成本：0 次模型调用；Lean 侧 1 个作业。

3.3 步骤 3：廉价 tactic 兜底

模块：扩展 `sgslean/SgsLean/Trivial.lean`，新增 `tryCheapTactics : String -> TacticM (Option String)`；在 `sgslean/SgsLean/Server.lean` 增加命令 `cheap`。

输入：`stmt`

输出：`{"hit": bool, "tactic": str, "proof": str}`；命中时 `proof` 就是该 tactic 本身。

算法：在孤立义务目标上依次尝试下表 tactic，任一条在预算内清空子目标即返回。

失败处理：全部未命中则进入步骤 4。

成本：0 次模型调用。

顺序	tactic 清单	说明
第一批	<code>rfl, decide, simp, norm_num, omega</code>	零参数、最便宜，已有 <code>Trivial</code> 模块的 <code>probes</code> 覆盖了其中三条，直接扩展
第二批	<code>ring, field_simp, positivity, linarith, nlinarith, constructor + aesop</code>	中等成本； <code>aesop</code> 依赖 <code>Mathlib</code>
第三批	<code>aesop</code> 加更大预算	最贵的一档，放在最后

心跳预算通过环境变量 `SGSLEAN_TRIVIAL_HEARTBEATS` 设置，标定值必须写进实验记录（它是“非平凡”判据的定义参数）。

3.4 步骤 4：分层检索

模块：新增 `sgsr/pipeline/retrieval.py`，函数 `retrieve(stmt, budget) -> List[Premise]`

第一层（精排层）：从我们自己的库 L 中取前 n_1 条，按 `score`（定义见 5.5 节）降序。实现先用廉价的符号重叠打分，库规模超过 200 条后再换成嵌入检索。

第二层（兜底层）：调用外部前提检索工具（`LeanSearch v2` 的 API，或自托管的 `LeanExplore`），取前 n_2 条。不自己训练检索器——这条赛道已被充分占据，自研期望收益低。

输出：`{"name", "statement", "source": "library" | "mathlib", "tokens"}`

预算控制：两类前提拼进提示词的总 token 数不得超过 `CTX_LEMMA_BUDGET`（默认 1200 token），超出则按 `score` 截断。

失败处理：外部检索不可用则跳过第二层并记录 `retrieval_degraded=true`；不得因此中断求解。

成本：检索 API 的调用若计费则记入成本，本地库检索记 0。

3.5 步骤 5：求解（生成候选证明）

模块：`sgsr/models/proxy.py` 的 `/solve`；客户端在 `prover.py` 内。

输入：`stmt`、检索到的前提、采样数 k 、温度、token 上限

输出：`{"proofs": [str], "usage": {...}, "model": str, "seed": int}`

算法：单次调用返回 k 篇候选（由 `solve_prompt` 要求模型输出多段以分隔符隔开的脚本，由 `extract_proofs` 解析）。若解析出的数量小于 k ，如实记录实际数量。

失败处理：后端不可用（503 或超时）则重试一次，仍失败则本轮判失败并记录 `backend_error`（不得把它当成“模型证不出”）。

成本：记 `prompt_tokens` 与 `completion_tokens`。

3.6 步骤 6：内核验证

模块：`sgslean/SgsLean/Verify.lean`，经 Lean 服务批量调用

输入：`(stmt, proof)` 对，一次批量提交 k 对

输出（每篇） : `{"ok": bool, "reason": str, "detail": str, "goalsLeft": int, "finalChecked": bool}`

算法：造孤立义务目标，用 `wrapProofScriptAsTactic` 包装脚本，执行后检查赋值中无 `sorry` 与残余元变量，最后交内核复核。

结构化的 reason 码表：`ok / parse_error / unknown_identifier / type_error / unclosed_goals / mvar_or_sorry / timeout / exception / final_check_failed`。

成本：Lean 墙钟；批处理时按批大小放大心跳预算（我们踩过这个坑）。

3.7 步骤 7：repair 定向重试

模块：新增 `sgsr/pipeline/repair.py`，函数 `repair(stmt, failed: List[Attempt], k) -> List[str]`

输入：原命题 + 上一轮的失败记录（命题、脚本、`reason`、`detail`）

输出：新一轮 k 篇候选

算法：把失败信息按固定模板拼进提示词（见 4.3 节的 REPAIR 区块），要求模型“针对这个错误重写”。若上一轮全部是 `mvar_or_sorry`，则在提示词中追加“不要使用 `sorry / admit`”的硬指令。

失败处理：达到轮数上限 R （默认 2）或 token 预算耗尽即停止。

成本：每轮记一次 `/solve` 的成本。

3.8 步骤 8：输出与记账

模块：`prover.py` 的 `Prover.prove`

输出：一个 `RunRecord`，字段见附录 A。含：是否解出、命中的步骤（兜底 / 求解 / `repair`）、通过的脚本、调用次数与 token、Lean 墙钟、库版本号。

硬规则：任何未通过步骤 6 的脚本一律不出现在 `proof` 字段中。

3.9 步骤 9：预算分配与停止条件

单题的 token 预算按级联分配，先便宜后贵：

$$B_{\text{total}} = B_{\text{retrieval}} + k \cdot B_{\text{solve}} + R \cdot k \cdot B_{\text{repair}}$$

默认取值： $k = 4$ ， $R = 2$ ，CTX_LEMMA_BUDGET=1200 token。停止条件是”某一步命中”，或”预算耗尽”。不许在没有预算上限的情况下无限重试——那样成本指标失去意义。

4 求解器规格

本节回答”用什么求解器、固不固定”。

4.1 默认配置（写死在配置文件里）

项	默认值	位置与说明
模型	deepseek-flash	sgsr/models/.env 的 DEEPSEEK_MODEL
后端	OpenAI 兼容 HTTP	sgsr/models/backend.py
温度（求解）	0.6	SOLVE_TEMPERATURE；求解要多样性
采样数 k	4	命令行参数；实验记录必须写入实际值
输出上限	4096 token	SOLVE_MAX_TOKENS
思考开关	关闭	DISABLE_THINKING=1。实测关闭后单次约 83 token，开启后需要 3k 至 10k token；作为成本杠杆在实验 C 组之外单独做一次对照
温度（出题）	1.0	CONJECTURE_TEMPERATURE
温度（评审）	0.0	GUIDE_TEMPERATURE，只在 A 组用到

4.2 求解器固定吗：分两层回答

代码层面：可插拔。证明后端定义为统一接口——”给定命题与前提，返回若干候选证明脚本”。第 4 步是接口调用点，任何实现该接口的东西都能接进来（本地 tactic、API 模型、外部 hammer、以后的搜索）。这样做的好处是正确性由步骤 6 的内核闸门保证，与后端无关。

实验层面：冻结。同一组对照（A/B/C）必须使用同一模型、同一温度、同一 k 、同一 token 上限；这些值写入每次运行的元数据。跨实验更换模型时，必须重新做难度标定（因为”近失手”的集合会整体漂移）。

不做事：不训练、不微调求解器。换更好的模型是”改一行配置”，在我们的指标里属于成本与正确率的取舍，不属于方法本身。

4.3 提示词规格

求解提示词由固定区块拼成，区块为空时整块不出现——这样才能构成干净的两臂对照：出题提示词同理，但把”命题”换成”未解目标”，并额外加入下面两个区块：

已踩过的坑：把需求裸列进提示词，模型会把需求原样抄回来当答案。因此需求区块的标题必须写明”这是背景证据，不是答案”，并配一条硬约束。

区块	内容与约束
ROLE	声明它是 Lean 4 证明助手，只输出 tactic 脚本
TARGET	待证命题（闭式）
AVAILABLE LEMMAS	从库里检索出的已证引理，带常量名。库为空时该区块整体不出现
MATHLIB HINTS	检索到的 Mathlib 前提，可选
CONSTRAINTS	不使用 <code>sorry</code> 与 <code>admit</code> ；只输出脚本本体
REPAIR	仅 repair 轮次出现：上一次的失败码与 Lean 错误原文

区块	内容与约束
BACKGROUND EVIDENCE (not the answer)	需求签名列表。标题必须写明这是背景证据而不是答案
SEEDS	库中已有引理的范例
HARD RULE	候选必须提到目标中出现的至少一个符号；不得原样复述需求

4.4 输出解析规则

`extract_proofs` 的规则（写死，不要每次改）：

1. 以代码块围栏（````lean4` 或 `````）为单位切分；
2. 去掉围栏与首尾空白；丢弃空块；
3. 丢弃含 `sorry` 或 `admit` 的块（但要记录”模型吐了 `sorry`”这一事件，因为它是模型能力的信号，我们实测占 6.4%）；
4. 保留顺序，最多取 k 个；
5. 若少于 k 个，如实记录实际数量，不用重复填充。

4.5 成本模型

每次调用的成本按 token 记为

$$\text{cost} = p_{\text{in}} \cdot T_{\text{prompt}} + p_{\text{out}} \cdot T_{\text{completion}}$$

其中单价 p 从配置文件读取（便于换模型时统一换算）。所有实验必须同时记录 **token** 数与折算金额，因为单价会变，token 数不会。

4.6 换模型的影响（写在文档里，避免临时争论）

- 换模型不需要改代码，只改配置。
- 换模型后必须重新标定难度： C 、 D 、 T 上每题的解出率都会变，”近失手”集合随之漂移。
- 换模型后此前所有库与结论作废，因为库的价值依赖于用它测出来的判据。

5 引理库的建立与管理（离线）

5.1 一轮闭环的伪代码

function BUILD_ROUND(C, L, config):

- (1) COLLECT solve all problems in C; record traces (subgoal signatures)
 - (2) DEMAND mine demands: $d(g) = \text{freq}(g) * \text{cost}(g)$; require $\geq m$ targets
 - (3) CONJECTURE for each unsolved target, condition on
 [target + demands + examples from L]; produce n candidates
 - (4) SCREEN gate: well-formed Prop; hard gates: nontrivial AND novel
 - (5) PROVE solve each candidate k_p times; ≥ 1 kernel-checked $\Rightarrow$ provable
 - (6) MEASURE two-arm measurement on W_{sel} ; compute reuse(1) and cost(1)
 - (7) SELECT density greedy under token budget $B_{\text{ctx}} \rightarrow$ new set S
 - (8) COMMIT materialize S , compile-check, append to L , bump library version
- return (updated L, round statistics)

5.2 需求挖掘

模块: sgsr/pipeline/demand.py（已实现，需按下列公式校准）

输入: 轨迹集合，每条轨迹含（题目 id, 是否解出, 子目标签名序列）

子目标签名定义: 上下文 $\vdash$ 目标，两部分都经空白折叠归一化（签名的定义参数是”是否含上下文”，我们用的是含上下文的 v1.1）。

公式:

$$d(g) = \text{freq}(g) \times \text{cost}(g), \quad \text{freq}(g) = \sum_{\text{题目 } w} \mathbf{1}[g \text{ 出现在 } w \text{ 的轨迹中}]$$

$\text{cost}(g)$ 取该子目标在轨迹中出现位置的平均序号（越靠后越”下游”，越难）。升级为”需求”的条件（三个同时满足）：(i) 出现在 $\geq m$ 个不同目标中，默认 $m = 2$ ；(ii) $d(g)$ 排名前 N_d ，默认 $N_d = 20$ ；(iii) 不是恒真或已被库中等价命题覆盖。

输出: $\{\text{"sig": str, "freq": int, "cost": float, "d": float, "targets": [id]}\}$ 列表，按 d 降序。

分层统计: 成功轨迹与失败轨迹分开统计，两边的 freq 都进报告（失败轨迹里的高频子目标更可能是真缺口）。

5.3 出题者

模块: sgsr/pipeline/conjecture.py + sgsr/models/prompts.py 的 conjecture_prompt

输入: 未解目标列表、需求签名、库中已有引理范例

输出: 候选引理列表，每条含 statement 与 source_target

参数: 每个目标出 n 条，默认 $n = 3$ ；温度 1.0。

三组实验的区别只在这一步的输入（见 8.1 节）：A 组只给未解目标；B 组给未解目标加需求；C 组给未解目标加随机等量的伪需求。

相关度过滤：候选必须提到目标中出现过的至少一个符号（廉价文本代理），否则标 `fail_irrelevant` 并由（不能混进”通过”）。

5.4 硬门

硬门	模 (SgsLean/ 下)	块 判定	预算/参数
合法	<code>Gate.lean</code>	elaborate 成 Prop	无
非平凡	<code>Trivial.lean</code>	decide/simp/aesop 在预算内解不出	SGSLEAN_TRIVIAL_HEARTBEATS, 默认 2×10^5
新颖	<code>Novelty.lean</code>	不与父目标等价，也不与库中已有引理等价（ α 等价 + 库检索未命中）	against 列表只放父目标与库，不许放全部目标（我们踩过这个坑：既违背语义又吃光预算）
可证	<code>Verify.lean</code>	采样 k_p 次，至少一篇过内核	k_p 默认 2

5.5 复用判据（本项目的核心）

这是本框架替换 SGS 的 Guide 的地方，也是唯一的方法性改动，因此写全。

5.5.1 要解决的问题

SGS 的 Guide 用 LLM 打分衡量”这条合成题对当前这道目标有没有用”。但本领域反复承认的痛点是”抽出的引理大多问题特异、不可复用”。判据和痛点是错位的：一个只优化”帮到当前目标”的信号，必然倾向于奖励问题特异的引理。

5.5.2 定义

设库 $L = \{\ell_1, \dots, \ell_m\}$ ，工作负载 W_{sel} (C 的子集， $|W_{sel}| = n_w$)。对每条引理 ℓ ：

$$\text{reuse}(\ell) = |\{w \in W_{sel} : \exists y \in Y_w, \text{verify}(w, y) = 1 \wedge \ell \in \text{consts}(y)\}|$$

即：在给库 L 的条件下解出的题目中，有多少不同题目的通过验收的证明实际引用了 ℓ 。consts(y) 由 Lean 侧从证明脚本抽取常量集合得到（复用 `Measure/Dependency.lean` 的依赖抽取能力）。

三个设计决定，必须照做：

1. 按”不同目标”计数，不按引用次数。一条引理被同一道题引用十次，记 1 次复用。理由是”可复用”的定义就是跨题。
2. 只统计通过验收的证明的引用。失败的尝试里出现某个名字不算复用，否则模型乱写名字就能刷分。

3. **必须做环境预检**: 先用一条最便宜的作业确认处理臂真的能 `import` 到库; 否则”处理臂”其实只是提示词里多了一段文字。这一步我们曾因为一个编译目标的坑得出过完全错误的结论。

成本定义为引理进入提示词的开销:

$$\text{cost}(\ell) = \text{token}(\text{render}(\ell))$$

即把 ℓ 的声明与说明渲染成提示词区块后的 token 数（用与后端一致的分词器估算）。

得分:

$$\text{score}(\ell) = \frac{\text{reuse}(\ell)}{\text{cost}(\ell)}$$

5.5.3 选择算法

目标: 在提示词预算内选一个子集, 使覆盖增益最大。

$$\max_{S \subseteq \mathcal{P}} \sum_{w \in W_{sel}} [r_S(w) - r_{\emptyset}(w)] \quad \text{s.t.} \quad \sum_{\ell \in S} \text{cost}(\ell) \leq B_{ctx}, \quad S \text{ 内每条过硬门}$$

其中 $r_S(w)$ 是给库 S 时题目 w 的通过率（通过数除以 k , 分母固定为 k , 没生成出证明也算失败), $r_{\emptyset}(w)$ 是无库基线。

- **理论上**: $r_S(w)$ 是并集形式的覆盖函数, 单调子模。在条数约束下, 标准贪心有 $1 - 1/e$ 的近似保证; 在预算约束下, 用”边际增益除以成本”的密度贪心并与最优单条取较优, 有 $\frac{1}{2}(1 - 1/e)$ 的保证。报告里要写明实际用的是哪一种。
- **工程上**: 真算边际增益需要对每条候选重跑两臂, 代价过高。第一阶段用 $\text{score}(\ell)$ 作为边际增益的代理, 并校验代理的忠实度: 随机抽 10 条候选, 同时计算真实的边际增益与代理值, 报告两者的 Spearman 相关系数。相关系数低于 0.5 则说明代理不可用, 改用其他代理（如依赖图 PageRank）。
- **预算口径**: 默认按 token 预算 B_{ctx} （因为上下文才是真实成本）, 同时报一份”按条数 B ”的结果便于与文献对照（DreamProver 用 100 条）。

5.5.4 淘汰规则

每 R_{evict} 轮（默认 3 轮）重跑一次 W_{sel} 上的复用测量, 然后:

1. **僵尸淘汰**: $\text{reuse}(\ell) = 0$ 且已入库超过 R_{evict} 轮 $\Rightarrow$ 移出并存入冷存文件（不删除, 保留来源）。
2. **低效降级**: $\text{score}(\ell)$ 低于全库中位数的一半 $\Rightarrow$ 降到冷存, 腾出预算。
3. **去重**: 新候选与库内已有引理的语句若归一化后相同, 或 α 等价, 直接丢弃（复用 `Novelty.lean`）。
4. **探索额度**: 新入库引理在前 R_{evict} 轮内不参与淘汰, 保证它有机会被测量到——否则新引理永远活不过第一轮, 库会退化成一个只增不减的僵化集合。

5.6 物化与入库

模块: `sgslean/SgsLean/Materialize.lean`（已实现）

输出文件: `sgslean/SgsLean/GeneratedLibrary.lean`（必须放在库内才能被 `import`）

命名规则: `sgs_lem_<序号>`，序号单调递增，永不复用。

必须紧接着执行: `lake build SgsLean.GeneratedLibrary`（模块目标，不能用库目标）。库目标 `lake build SgsLean` 不会编译这个模块，这曾导致“处理臂其实没有库”的假结论。

库记录文件: `sgs-reap/experiments/library.jsonl`，每行一条，字段见附录 A。

版本快照: 每次入库后写 `library_v<N>.jsonl` 并记录文件内容的哈希，作为本次实验的库版本号。

6 有没有要训练的小规模参数

答案: 没有。可训练参数为 0。

明确列出不训练的东西，以及不训练的理由（避免以后反复讨论）:

不做事	理由	替代方案
微调 / 强化学习求解器	需要 GPU 与大规模数据, 超出资源约束; 且本项目的贡献定位是“模型之上的脚手架”	换更好的 API 模型（改一行配置）
训练出题者	同上	用提示词条件化（需求 + 库范例）
训练评审者（Guide）	同上	改用可计算的形式化判据（硬门 + 复用判据），这是本项目的核心改动
训练检索器	已有成熟公开系统	直接调用 LeanSearch v2 API 或自托管 LeanExplore
训练难度预测器	会引入额外不确定性	用实测解出率直接分档

预留接口（只留位，不实现）: 如果将来拿到 GPU，第一个要训练的对象是**出题者**，奖励为 $R = R_{\text{可证硬门}} \times \text{score}(\ell)$ ，即以“复用除以成本”为奖励信号——这正好是 SGS 的做法换一个奖励。代码上只需在 `pipeline/` 下留一个 `train_conjecturer.py` 的空壳与数据导出函数，数据格式沿用第 5 节的候选记录，不需要改动其他模块。

集合	用途	规模	来源要求
C	建库：采轨迹、挖需求、出引理、复用测量	100 至 300 道	单一领域的 来源甲 。不得与 D 、 T 有任何重叠或同源关系
D	调参：定 k 、阈值、库容；看方向	50 至 150 道	同领域的 来源乙
T	最终报告	50 至 150 道	来源乙，与 D 切分且不相交；冻结后只跑一次

7 数据集规格

7.1 来源与用途

7.2 C 的来源与同源检查

选一个领域（建议**不等式或组合**）， C 取自该领域的训练划分，凡被测试基准覆盖的数据集一律排除。执行前必须跑一遍检查脚本：

- 命题级查重**： C 中每条命题与 $D \cup T$ 做归一化后的文本比对，命中即丢弃；
- 来源级排查**：记录每条数据的原始出处，若 C 与 $D \cup T$ 的出处存在聚合关系（例如都派生自同一份聚合语料），必须抽样人工核对；
- 泄漏标记**：每道题带 `source` 字段，实验记录里保留，便于事后审计。

7.3 难度标定流程（必须先做）

”近失手”的题目才是有信号的地方：太难（解出率 0）两臂皆零，太易（解出率 1）不需要引理。因此每换一次模型或 k ，都要重新标定：

```
function CALIBRATE(dataset, solver, k):
  for each problem w:
    run solver k times; verify each attempt
    rate(w) in {0, 1/k, ..., 1}
  buckets:
    impossible    rate == 0           # no signal: both arms give 0
    near_miss     0 < rate < 1        # the only informative band
    easy          rate == 1           # used only as a non-degradation check
  return per-problem rates and bucket assignment
```

档位	判据	用途
impossible	解出率 = 0	不用于测增益（会对两臂都给出 0，无信息）
near_miss	0 < 解出率 < 1	主要测量对象；据实测约占一成
easy	解出率 = 1	只用于”给库不会变差”的反向对照

7.4 测试集 T 是什么

不使用 **miniF2F** 作为主测试集。理由：该基准已被做到 96% 至 99%，天花板效应会吞掉一切真实增益。它最多作为“不退化”的旁证出现。

候选集合（按优先级，均为公开的、有难度的分域集合）：

名称	规模	领域	备注
567NEQ	92	不等式	首选。中等规模，领域单一，与 C 来源分离
ChenNEQ	42	不等式	与 567NEQ 同族，可合并或作交叉验证
MO-INT	20	不等式	规模小，只作补充
CombiBench 子集	43	组合	若主领域改为组合则用这个
ProofNet	371	本科数学	域更宽，可作为跨域旁证
PutnamBench	660	本科竞赛	谨慎： 不同论文报告的数字差异达八倍，说明协议不一致，只用其子集并写清协议

T 的确定规则（写死为可执行判据）：先用基线求解器在候选集合上跑 $k = 3$ 的标定，保留目标级非零解率落在 $[0.15, 0.85]$ 的题目构成 T ，规模控制在 50 至 150 道。

- 非零解率高于 0.85：太易，无余量，弃用；
- 低于 0.15：太难，两臂都测不出信号，弃用；
- 若某候选集合里合格题目不足 50 道，则合并其他集合，或退化为自建集合——做法是拿更大语料用基线筛出近失手题，并把筛选题与 C 、 D 严格隔离。

7.5 防泄漏检查清单（每次正式实验前逐条打勾）

1. $C \cap (D \cup T) = \emptyset$ ，且来源无聚合关系；
2. 库中没有任何引理的 `source_target` 落在 $D \cup T$ 上；
3. 需求挖掘只在 C 上做过；
4. 所有超参数（ k 、 B_{ctx} 、 m 、 N_d 、阈值）都在 D 上确定，并写进报告；
5. T 的运行计数为 1，且运行前已冻结库版本与代码提交号。

8 实验设计

8.1 三组对照

同一求解器、同一 k 、同一课程集、同一工作负载、固定分母，只改出题者的条件化信号与审判据：

三组必须完全同预算：同样的 k 、同样的候选数 n 、同样的模型调用上限。若某组因为判据更贵而超支，必须在报告里按等成本截断后再比较。

组	出题者输入	评审判据	对应
A	未解目标	LLM 打分 (SGS 的 rubric)	SGS 原样; /guide 与 rubric 已完整保留
B	未解目标 + 需求签名	复用 ÷ 成本	本框架
C	未解目标 + 随机等量伪需求	复用 ÷ 成本	消融: 排除”增益只是来自多给了上下文”

8.2 指标与公式

指标	定义
目标级 $\text{pass}@k$	见 1.2 节公式; 在 T 上只算一次
复用率	$\frac{\#\{\ell \in L : \text{reuse}(\ell) \geq 2\}}{ L }$ (被至少两个不同目标引用的引理占比)
僵尸率	$\frac{\#\{\ell \in L : \text{reuse}(\ell) = 0\}}{ L }$
平均复用度	$\frac{1}{ L } \sum_{\ell} \text{reuse}(\ell)$
CostPerSolved	总 token 除以解出题数
CostPerReusable	建库 token 除以可复用引理数 (核心成本指标)
无库 / 有库增量	$\text{pass}@k_L - \text{pass}@k_{\emptyset}$, 两臂同 k 、同题、同模型

8.3 样本量与统计功效（先算再跑）

在 $|T|$ 道题、每题 k 次采样下，解出率的估计标准误约为 $\sqrt{p(1-p)/|T|}$ 。取 $|T| = 120$ 、 $p = 0.4$ ，则一个标准误约 4.5 个百分点。因此：

- 我们能够分辨 10 个百分点以上的差异；
- 我们不能分辨 3 个百分点以下的差异——这类差异一律报为”无显著差异”；
- 若要分辨更小的差异，必须增大 $|T|$ 或对同一集合做重复测量并报置信区间。

这条要写进报告，避免把噪声当结论。同时提醒：题目级通过率是相关的，上述公式是乐观下界，实际置信区间应更宽（建议用自举法给出）。

8.4 预算与墙钟估算

以 $|C| = 300$ 、 $|W_{\text{sel}}| = 150$ 、 $|T| = 120$ 、 $k = 4$ 、建库 3 轮、 $n = 3$ 估算：

按每次调用平均 1.5k token 估算，总 token 约 1.4 千万，按当前 API 单价折算在几十元量级，远低于 500 元的单次实验上限。

真正的瓶颈是 Lean 墙钟：按实测约 2.8 秒每篇（批处理摊销了导入），8000 篇约 6 小时，加上 Mathlib 导入与批间隙，估计 8 到 10 小时。因此计划里允许把一次完整循环拆到两天执行，中间落盘中间结果。

环节	模型调用	Lean 验证	备注
C 上难度标定 ($k = 3$)	900	900	一次性
三轮采集轨迹	2700	2700	每轮在 C 上跑一遍
出题 (每轮 $n = 3$)	约 1350	0	只对未解目标出题
可证硬门 ($k_p = 2$)	约 2700	约 2700	每条候选两条
复用两臂测量 ($k = 4$)	约 1200	约 1200	W_{sel} 上两臂
T 上最终评测	约 480	约 480	只跑一次
合计	约 9300 次	约 8000 次	—

8.5 报告规范

- 报告写入 `sgs-reap/experiments/results/`，文件名带模式与规模后缀，例如 `abc_nearmiss_k4.json`；
- 每个报告必须含：库版本哈希、代码提交号、模型名与参数、 k 、token 总量、逐题解出情况、三条成本指标、复用分布直方图；
- 小规模试跑不许覆盖正式报告（用不同后缀）。

9 分阶段实施计划

每阶段给出：目标、要新增或修改的文件、函数签名、验收命令、闸门。

9.1 阶段 P1：证明器本体

目标：把”输入一条命题、输出一篇通过内核检验的证明”这件事跑通。这是大创交付物的主干，也是后续所有实验的入口。

验收命令：

```
python scripts/prove.py --statement "forall (n : Nat), n + 0 = n" --k 4
python scripts/prove.py --file tests/examples/t1.lean --k 4
python scripts/run_prover_eval.py --set D --k 4 \
    --out experiments/results/p1_dev.json
```

闸门：在 D 的 20 道上跑出第一组 `pass@k` 与 token 成本；兜底命中率单独报（它是不花钱的那部分）。

9.2 阶段 P2：小实验台

目标：把建库闭环在小规模上真正跑通，产物是一个能长大的库。

验收命令：

文件	动作	内容
sgsr/pipeline/prover.py	新增	class Budget、class Attempt、class ProofResult; class Prover, 方法 prove(stmt, budget) -> ProofResult; 内部按第 3 节九个步骤串起来
sgsr/pipeline/retrieval.py	新增	retrieve(stmt, budget) -> List[Premise]; 第一层库检索, 第二层外部检索, 含 token 截断与降级
sgsr/pipeline/repair.py	新增	repair(stmt, failed, k) -> List[str]
sgsr/models/prompts.py	修改	新增 repair_prompt(stmt, failed, k), 输出带 REPAIR 区块
sgslean/SgsLean/Trivial.lean	修改	新增 tryCheapTactics, 返回命中的 tactic 脚本（而不是 Bool）; 扩展 tactic 清单为三批
sgslean/SgsLean/Server.lean	修改	新增 cheap 命令
scripts/prove.py	新增	命令行入口: --file 或 --statement, --k, --library, --endpoint, --out
scripts/run_prover_eval.py	新增	在给定集合上批量跑, 输出 pass@k 与三条成本指标

文件	动作	内容
tools/prepare_domain_corpus.py	新增	把选定领域的多来源题目转成统一 jsonl; 做命题级去重与同源检查; 输出 C、D、T 三个文件
scripts/calibrate_difficulty.py	新增	调用求解器标定每题解出率, 分档, 输出近失手清单
sgsr/pipeline/coverage.py	修改	新增 reuse_cost_greedy(pool, ctx_budget) -> List[str]: 按 reuse/cost 的密度贪心, 并与最优单条取较优
sgsr/pipeline/demand.py	修改	按 5.2 节的公式与阈值校准; 输出带 targets 字段
sgsr/pipeline/conjecture.py	修改	条件化信号按 5.3 节扩展; 相关度过滤接入
sgsr/pipeline/runner.py	修改	把选择层从 parent_cover_greedy 换成 reuse_cost_greedy; 加入复用测量与淘汰步骤

```
python scripts/calibrate_difficulty.py --set C --k 3 \
    --out experiments/results/calib_C.json
python scripts/run_round.py --rounds 3 --curriculum data/C.jsonl \
    --k 2 --n 3 --ctx-budget 1200
python scripts/run_gate_g3_real.py --select nearmiss --limit 6 --k 4
```

闸门：库规模达到 20 条以上；两臂测量能给出非零的复用计数（若复用计数全为 0，说明库或检索没有生效，先查环境预检与检索层，不要继续）。

9.3 阶段 P3：三组对照

目标：拿到 A/B/C 的对照数据。

文件	动作	内容
scripts/run_ablation_abc.py	新增	按 8.1 节跑三组，每组产出库、复用分布、成本与 pass@k；强制写入库版本与提交号
sgsr/pipeline/runner.py	修改	支持 --conditioning {target,demand,random} 与 --judge {llm,reuse} 两个开关

验收命令：

```
python scripts/run_ablation_abc.py --group A --rounds 3 --k 2 --n 3
python scripts/run_ablation_abc.py --group B --rounds 3 --k 2 --n 3
python scripts/run_ablation_abc.py --group C --rounds 3 --k 2 --n 3
```

闸门：三组都得到可比较的同一格式报告；若 A 组因 LLM 评审成本膨胀而超预算，按等成本截断后再比。

9.4 阶段 P4：成本与复用分析

目标：把结果整理成论文的核心图表。

文件	动作	内容
scripts/analyze_costs.py	新增	输出：三组的成本 Pareto 曲线、三条成本指标对比、复用分布直方图、僵尸率随轮次变化曲线
scripts/audit_labels.py	新增	审计“引用计数”这一标签的可靠性（抽样人工核对），因为检索类工作已证明这类代理标签可能只有约一半吻合

闸门：产出论文主图（复用率对成本）与主表（三组对比）。

9.5 阶段 P5：写作

大创报告 + 论文短文。论文的骨架已经定好：问题（判据与痛点错位）、方法（复用判据 + 需求条件化）、对照（A/B/C）、结果（三组在同等预算下的复用率与成本）、限制。

10 风险与降级预案

风险	征兆	降级方案
复用计数始终为 0	两臂测量里没有一条引理被引用	先查环境预检与检索层（这是工程问题，不是结论）；仍为 0 则说明”提示词注入不被使用”，转为研究该现象本身
三组无差异	A/B/C 的复用率与 $\text{pass}@k$ 都在噪声内	报告为”条件化信号不影响结果”的实证结论，并把成本 Pareto 作为主结果
库长不起来	硬门淘汰率过高（如非平凡率极低）	放宽非平凡预算；或改用难度更低的课程集让模型能产出可证的引理
Lean 墙钟失控	单轮超过 4 小时	缩小 C 与 W_{sel} ；确认批处理只付一次 Mathlib 导入
外部检索不可用	第二层检索超时	只用第一层（自建库）跑完全部实验，并在报告里注明

贯穿始终的底线：无论实验结果如何，证明器本体与一套可复现的成本账都是完整的大创交付物。论文部分允许负结果。

A 数据结构（字段级）

A.1 运行记录 RunRecord

```
{
  "run_id": "20260921T101500Z",
  "commit": "abc1234",
  "model": "deepseek-flash",
  "temperature": 0.6,
  "k": 4,
  "library_hash": "sha256:...",
  "library_size": 37,
  "stmt": "forall (n : Nat), n + 0 = n",
  "solved": true,
  "path": "cheap|solve|repair",
  "proof": "intro n; rfl",
  "attempts": [{"proof": "...", "reason": "type_error", "detail": "..."}],
  "usage": {"prompt_tokens": 812, "completion_tokens": 143},
  "lean_ms": 2870,
  "wall_ms": 4120
}
```

A.2 轨迹记录 Trace

```
{
```

```

    "target": "aime_1988_p3",
    "solved": false,
    "k": 3,
    "subgoals": [
      {"sig": "x : Real, h : 0 < x |- Real.logb 2 x = Real.logb 8 x * 3",
       "pos": 3, "from": "failed_attempt_2"}
    ]
  }
}

```

A.3 需求记录 Demand

```

{
  "sig": "...",
  "freq": 7,
  "cost": 3.4,
  "d": 23.8,
  "targets": ["aime_1988_p3", "amc12a_2016_p3"]
}

```

A.4 库条目 LibraryEntry

```

{
  "name": "sgs_lem_12",
  "stmt": "forall (x : Real) (h : 0 < x), Real.logb 2 x = Real.logb 8 x * 3",
  "proof": "intro x h; ...",
  "verified": true,
  "source": "cand:aime_1988_p3#0",
  "source_corpus": "C",
  "reuse": 3,
  "cost_tokens": 42,
  "score": 0.0714,
  "added_round": 2,
  "status": "active|cold|evicted",
  "library_version": "v7"
}

```

B 代码骨架（函数签名清单）

以下签名是实现时的最低要求，命名可微调但语义不得更改。

```

# sgsr/pipeline/prover.py
class Budget: total_tokens:int; k:int; repair_rounds:int; ctx_lemma_tokens:int

```

```

class Attempt: proof:str; reason:str; detail:str
class ProofResult: solved:bool; proof:str|None; path:str
                  attempts:list; usage:dict; lean_ms:int
class Prover:
    def __init__(self, endpoint:str, backend:str, library_path:str|None)
    def prove(self, stmt:str, budget:Budget) -> ProofResult

# sgsr/pipeline/retrieval.py
class Premise: name:str; statement:str; source:str; tokens:int; score:float
def retrieve(stmt:str, lib:list[LibraryEntry], budget:int) -> list[Premise]
def retrieve_mathlib(stmt:str, n:int, timeout_ms:int) -> list[Premise]

# sgsr/pipeline/repair.py
def repair(stmt:str, failed:list[Attempt], k:int, endpoint:str) -> list[str]

# sgsr/pipeline/coverage.py
def reuse_cost_greedy(pool:list[LibraryEntry], ctx_budget:int) -> list[str]
def two_arm_measure(workload:list[str], lib:list[LibraryEntry], k:int) -> dict
def marginal_gain(workload:list[str], S:list[LibraryEntry],
                  cand:LibraryEntry, k:int) -> float

# sgsr/pipeline/runner.py
class RoundRunner:
    def run_round(self, round_idx:int) -> dict
    def run(self, rounds:int) -> dict

# scripts/prove.py, scripts/run_prover_eval.py, scripts/calibrate_difficulty.py,
# scripts/run_ablation_abc.py, scripts/analyze_costs.py （命令行入口）

```

C 术语与缩写

术语	含义
SGS	本项目所依据的自博弈框架（求解者、出题者、评审者三角色）
硬门	候选引理入库前必须通过的布尔判据：合法、非平凡、新颖、可证
复用判据	本框架的核心： $\text{reuse}(\ell)/\text{cost}(\ell)$ ，用于选择、排序与淘汰
僵尸引理	入库后从未被任何目标引用的引理
近失手	解出率严格介于 0 与 1 之间的题目；唯一有增益信号的区间
两臂测量	同求解器、同 k ，分别在有库与无库条件下测量通过率
环境预检	用一条最便宜的作业确认处理臂真的能 import 到库